

Lineare Algebra - Übungen 8

Klaus Rheinberger, FH Vorarlberg

15. April 2025

i Abgabe in ILIAS

- Alle PYTHON-Aufgaben gesamthaft in einem lauffähigen Jupyter Notebook, siehe Vorlage in ILIAS.
- Alle PAPIER-Aufgaben gesamthaft in einer PDF-Datei inkl. Name auf den gescannten Seiten.

1 Müllmengen - Teil 2 (2 + 3 = 5 Punkte)

Die Müllmenge in Millionen Tonnen pro Tag des Landes [Lower Slobbovia](#) von 1960 bis 1995 ist in folgender Tabelle wiedergegeben.

Jahr t	1960	1965	1970	1975	1980	1985	1990	1995
Menge y	86.0	99.8	135.8	155.0	192.6	243.1	316.3	469.5

1. PAPIER: Formulieren Sie das OLS-Problem in Matrixform für den besten Fit eines exponentiellen Wachstumsmodells $y = ce^{\alpha t}$ durch die Datenpunkte.
2. PYTHON: Lösen Sie das OLS-Problem, und stellen Sie Daten und optimalen Fit in einem Plot grafisch dar.

Lösung: Logarithmieren von $y = ce^{\alpha t}$ liefert $\ln(y) = \ln(c) + \alpha t$.

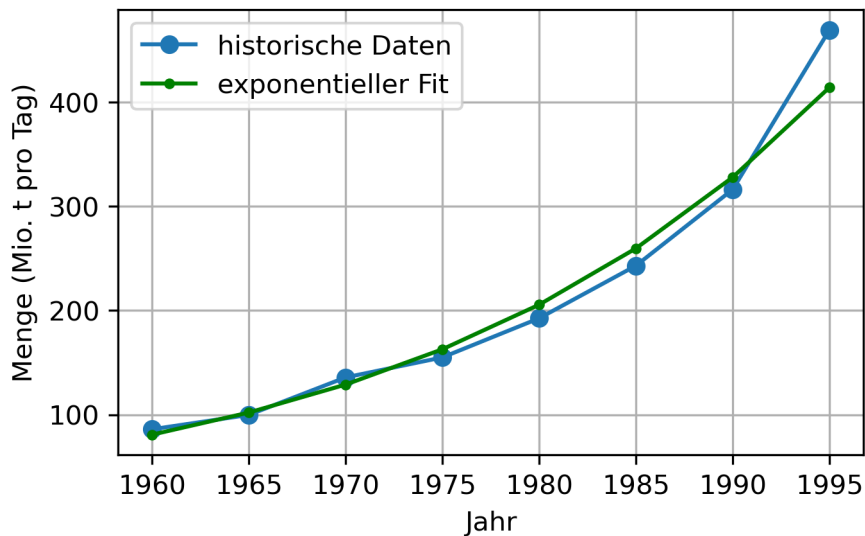
$$\min \left\| \begin{pmatrix} 1 & 1960 \\ 1 & 1965 \\ \vdots & \vdots \\ 1 & 1995 \end{pmatrix} \begin{pmatrix} \ln(c) \\ \alpha \end{pmatrix} - \begin{pmatrix} \ln(86.0) \\ \ln(99.8) \\ \vdots \\ \ln(469.5) \end{pmatrix} \right\|$$

```
t = np.arange(1960, 2000, 5)
y = np.array([86.0, 99.8, 135.8, 155.0, 192.6, 243.1, 316.3, 469.5])

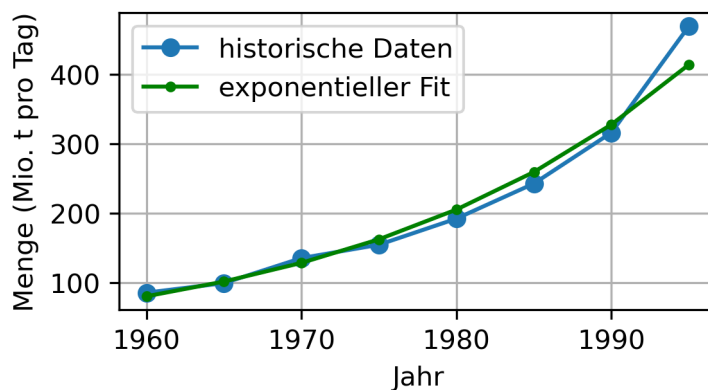
n = len(t)
A = np.column_stack( (np.ones((n, 1)), t) )
x_exp = np.linalg.lstsq(A, np.log(y))[0]
y_fit_exp = np.exp(A @ x_exp)

plt.figure(figsize=(5, 3))
```

```
plt.plot(t, y, 'o-', label='historische Daten')
plt.plot(t, y_fit_exp, '-g', label='exponentieller Fit')
plt.xlabel('Jahr')
plt.ylabel('Menge (Mio. t pro Tag)')
plt.legend(loc='best', numpoints=1)
plt.grid(True)
```



Endergebnis



2 Polynomiale Regression (1 + 2 + 1 + 2 = 6)

Laden Sie mit folgenden Python-Befehlen die Daten der Datei [Polyfit.csv](#) in ein array D, dessen Spalten als Daten-Vektoren t und y weiterverwendet werden:

```
D = np.genfromtxt('../daten/Polyfit.csv', delimiter=';')
y = D[:, 1]
t = D[:, 0]
```

Passen Sie zuvor den Pfad auf Ihrem Rechner zur Datei `Polyfit.csv` an.

1. PYTHON: Plotten Sie y gegen t .
2. PYTHON: Verwenden Sie die Methode der kleinsten Fehlerquadrate (Regression, Python-Befehl `lstsq`), um ein Polynom vom Grad 4 durch die Punktwolke zu fitten. Erzeugen Sie ein Diagramm der Koeffizienten. Warum sind die ungeraden Koeffizienten null?
3. PYTHON: Plotten Sie die ursprünglichen Datenpunkte und den polynomialen Fit in eine Grafik.
4. PYTHON: Benutzen Sie die Befehle `numpy.polyfit` und `numpy.poly1d` um die gleichen Resultate zu erzielen.

Lösung:

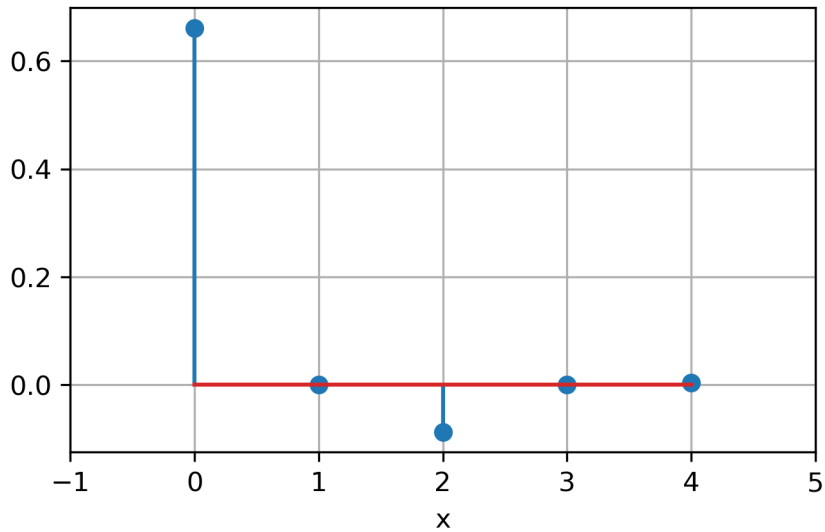
```
D = np.genfromtxt('../daten/Polyfit.csv', delimiter=';')
t = D[:, 0]
y = D[:, 1]
n = len(t)

A = np.column_stack( (np.ones((n, 1))), t, t**2, t**3, t**4 )
x = np.linalg.lstsq(A, y)[0]

for k in range(len(x)):
    print(f"Koeffizient der Ordnung {k} = {x[k]:10.6f}")

plt.figure(figsize=(5, 3))
plt.stem(x)
plt.xlabel('x')
plt.xlim(-1, 5)
plt.grid(True)
```

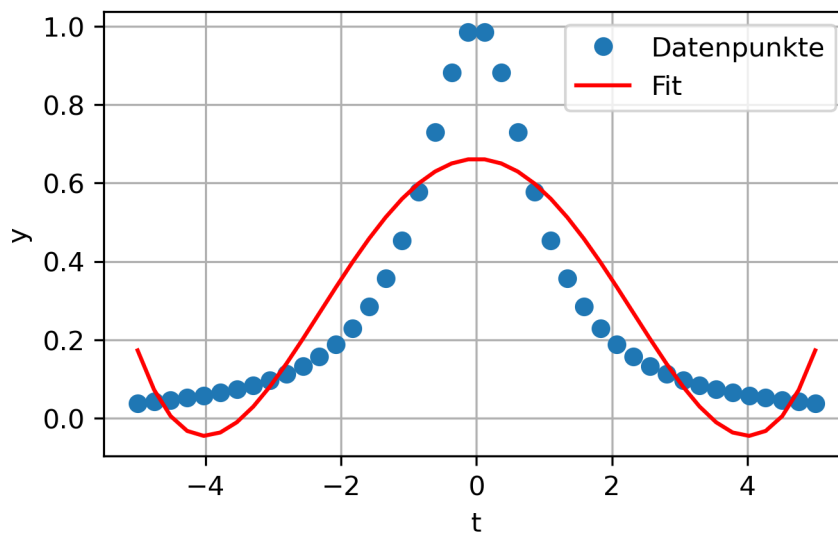
```
Koeffizient der Ordnung 0 =  0.661640
Koeffizient der Ordnung 1 = -0.000000
Koeffizient der Ordnung 2 = -0.087919
Koeffizient der Ordnung 3 =  0.000000
Koeffizient der Ordnung 4 =  0.002735
```



Plot des polynomialen Fits:

```
y_fit = A@x

plt.figure(figsize=(5,3))
plt.plot(t, y, 'o', label='Datenpunkte')
plt.plot(t, y_fit, '-r', label='Fit')
plt.xlabel('t')
plt.ylabel('y')
plt.legend(numpoints = 1)
plt.grid(True)
```



Befehle `polyfit` und `poly1d`:

```

ordnung = 4
c = np.polyfit(t, y, ordnung)

for k in range(ordnung + 1):
    print(f"Koeffizient der Ordnung {ordnung - k} = {c[k]:10.6f}")

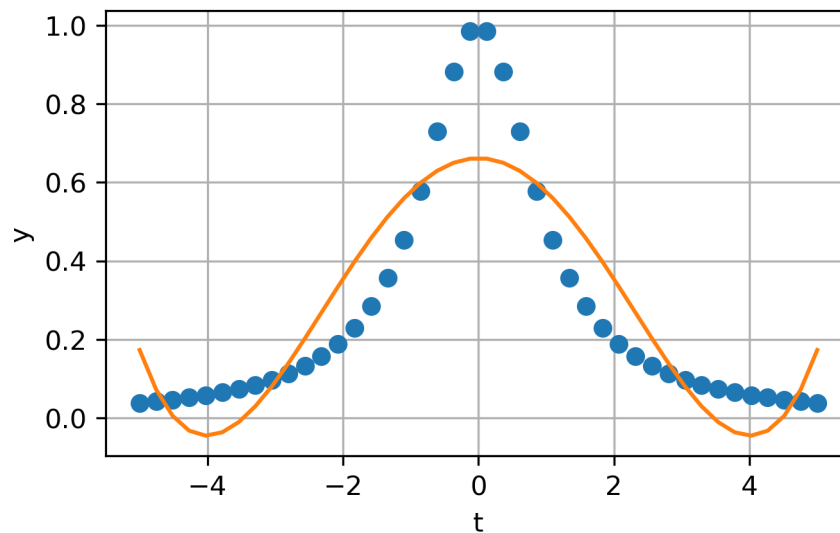
plt.figure(figsize=(5, 3))
plt.plot(t, y, 'o')
plt.plot(t, np.poly1d(c)(t) )
plt.xlabel('t')
plt.ylabel('y')
plt.grid(True)

```

```

Koeffizient der Ordnung 4 = 0.002735
Koeffizient der Ordnung 3 = 0.000000
Koeffizient der Ordnung 2 = -0.087919
Koeffizient der Ordnung 1 = -0.000000
Koeffizient der Ordnung 0 = 0.661640

```



Endergebnis

